

P0571r0

Type Requirements for <numeric> Algorithms

Published Proposal, 2 February 2017

This version:

<http://wg21.link/P0571r0>

Author:

[Bryce Adelstein Lelbach](#) (Lawrence Berkeley National Laboratory)

Audience:

SG1, LEWG, LWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Table of Contents

- 1 Overview
- 2 Existing <numeric> Algorithms
- 3 New Parallel <numeric> Algorithms
- References
- Informative References

§ 1. Overview

At the Issaquah 2016 meeting, during the LWG review of [\[P0452r0\]](#), it was realized that some of the numeric algorithms (both old algorithms and new ones from the Parallelism TS) had insufficient or unclear type requirements.

Consider:

```
vector<double> v;  
double d = accumulate(v.begin(), v.end(), 0);
```

What type is used to store the intermediary result during accumulation in the above example? Is it `T`, the type of the initial value (which is `int` here)? Or is it `double`, the value type of the `InputIterators`?

The answer: it is `T`, or `int` in the above example. For algorithms which do not take an initial value (such as `partial_sum` and `adjacent_difference`) and instead write to an output sequence via an `OutputIterator`, the value type of the `InputIterator` is used. This may not be the best or most consistent behavior, but it appears to have been the status quo in major implementations.

However, the specification lacks clarity about which types are used for accumulation and the constraints on the template parameters to the algorithms. This paper identifies some of the issues and proposes potential solutions.

§ 2. Existing <numeric> Algorithms

`accumulate`, `inner_product`, `partial_sum` and `adjacent_difference` are all specified using similar "accumulator-style" wording:

`accumulate` (26.8.2 [**accumulate**] paragraph 1):

Effects: Computes its result by initializing the accumulator `acc` with the initial value `init` and then modifies it with `acc = acc + *i` or `acc = binary_op(acc, *i)` for every iterator `i` in the range `[first, last)` in order.

Requires: `T` shall meet the requirements of `CopyConstructible` (Table 22) and `CopyAssignable` (Table 24) types. In the range `[first, last]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.

`inner_product` (26.8.5 [**inner.product**] paragraph 1):

Effects: Computes its result by initializing the accumulator `acc` with the initial value `init` and then modifying it with `acc = acc + (*i1) * (*i2)` or `acc = binary_op1(acc, binary_op2(*i1, *i2))` for every iterator `i1` in the range `[first1, last1)` and iterator `i2` in the range `[first2, first2 + (last1 - first1))` in order.

Requires: `T` shall meet the requirements of `CopyConstructible` (Table 22) and `CopyAssignable` (Table 24) types. In the ranges `[first1, last1]` and `[first2, first2 + (last1 - first1)]` `binary_op1` and `binary_op2` shall neither modify elements nor invalidate iterators or subranges.

`partial_sum` (26.8.6 [**partial.sum**] paragraph 1 and 4)

Effects: For a non-empty range, the function creates an accumulator `acc` whose type is `InputIterator`'s value type, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1, last)` in order, `acc` is then modified by `acc = acc + *i` or `acc = binary_op(acc, *i)` and the result is assigned to `*(result + (i - first))`.

Requires: `InputIterator`'s value type shall be constructible from the type of `*first`. The result of the expression `acc + *i` or `binary_op(acc, *i)` shall be implicitly convertible to `InputIterator`'s value type. `acc` shall be writable (24.2.1) to the result output iterator. In the ranges `[first, last]` and `[result, result + (last - first)]` `binary_op` shall neither modify elements nor invalidate iterators or subranges.

`adjacent_difference` (26.8.11 [**adjacent.difference**] paragraph 1 and 2)

Effects: For a non-empty range, the function creates an accumulator `acc` whose type is `InputIterator`'s value type, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1, last)` in order, creates an object `val` whose type is `InputIterator`'s value type, initializes it with `*i`, computes `val - acc` or `binary_op(val, acc)`, assigns the result to `*(result + (i - first))`, and move assigns from `val` to `acc`.

Requires: `InputIterator`'s value type shall be `MoveAssignable` (Table 23) and shall be constructible from the type of `*first`. `acc` shall be writable (24.2.1) to the result output iterator. The result of the expression `val - acc` or `binary_op(val, acc)` shall be writable to the result output iterator. In the ranges `[first, last]` and `[result, result + (last - first)]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.

The description of these four algorithms shares a similar structure:

- Create an accumulator `acc` which is initialized with `init` or `*first`.

- For (the) iterator(s) in the range(s) **in order**, modify `acc` by applying an update operation which takes `acc` and the dereferenced value(s) of the iterator(s) as arguments.

For the numeric algorithms which do not take an initial value (`partial_sum` and `adjacent_difference`), the accumulator `acc`'s type is the `InputIterator`'s value type.

For the numeric algorithms which do take an initial value (`accumulate` and `inner_product`), it is not entirely clear what the type of the accumulator should be, although it is strongly implied that it should be the type of the initial value (`T`) not the `InputIterator`'s value type. The return value of these two algorithms is `T`, and the *Requires*: clauses for both algorithms specify that `T` must be `CopyConstructible` and `CopyAssignable`, which would only make sense if the accumulator is to be of type `T`. In `libstdc++` and `libc++`, the `init` parameter is used as the accumulator (e.g. the accumulator type is `T`).

I suggest that we change `accumulate` and `inner_product` to

- Specify that the accumulator type is `T`.
- Require the result of the update expression for these algorithms (`acc + *i` or `binary_op(acc, *i)` for `accumulate`, for example) be implicitly convertible to `T`. `partial_sum` and `adjacent_difference` already have similar language requiring implicit convertibility to `InputIterator`'s value type.

§ 3. New Parallel `<numeric>` Algorithms

For the parallel numeric algorithms (`reduce`, `transform_reduce`, `inclusive_scan`, `exclusive_scan`, `transform_inclusive_scan` and `transform_exclusive_scan`), things are a bit more complicated. All of these algorithms are specified in terms of `GENERALIZED_SUM` or `GENERALIZED_NONCOMMUTATIVE_SUM`:

26.2 [`numeric.defns`]

Define `GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aN)` as follows:

- `a1` when `N` is 1, otherwise
- `op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, ..., aN))` for any `K` where $1 < K + 1 = M \leq N$.

Define `GENERALIZED_SUM(op, a1, ..., aN)` as `GENERALIZED_NONCOMMUTATIVE_SUM(op, b1, ..., bN)` where `b1, ..., bN` may be any permutation of `a1, ..., aN`.

Arbitrary partition of the reduction calculations at the implementation's discretion is allowed for `GENERALIZED_NONCOMMUTATIVE_SUM` algorithms (`exclusive_scan`, `inclusive_scan`, `transform_exclusive_scan` and `transform_inclusive_scan`). In addition to arbitrary partitioning, `GENERALIZED_SUM` algorithms (`reduce` and `transform_reduce`) can arbitrarily re-order the reduction calculations. All of the algorithms have overloads that take an initial value of type `T`. For the `GENERALIZED_SUM` algorithms, all overloads take an initial value of type `T`.

These algorithms present a conundrum for us. Should they:

- break the precedence of their serial counterparts (`accumulate`, `inner_product` and `adjacent_difference`) and use the value type of `InputIterator` instead of `T` for intermediate storage (the implied status quo of the working text, given the current definitions of `GENERALIZED_SUM` and `GENERALIZED_NONCOMMUTATIVE_SUM`).
- OR should the definition of algorithms be changed to require conversions to `T`, making the `GENERALIZED_SUM` algorithms consistent with their serial counterparts?

The second option seems very messy. We'd have to change the definition of `GENERALIZED_SUM` to explicitly take the initial value and initial value type into account. The second option would potentially force conversions to `T` that would be unnecessary and undesirable. Additionally, what about the algorithms which can take an initial value, but also have overloads which do not take one (`inclusive_scan` and `transform_inclusive_scan`)? The first option seems best and appears to be closest to the design intent.

I suggest that we change all the parallel numeric algorithms to:

- Require that the type of the initial value (`T`) be `CopyConstructible` (and `CopyAssignable?`).
- Require that `InputIterator`'s value type can be constructed from the type of the initial value (`T`).
- Replace uses of `init` in the `GENERALIZED_SUM` or `GENERALIZED_NONCOMMUTATIVE_SUM` expression used to define the algorithms with a temporary of the `InputIterator`'s value type constructed from the initial value.
- Require that the result of the `GENERALIZED_SUM` or `GENERALIZED_NONCOMMUTATIVE_SUM` expression be implicitly convertible to the correct return type for the algorithm (`T`).

Additionally, for the parallel numeric algorithms which take `OutputIterators`, I suggest the following changes:

- Require that the result of the `GENERALIZED_SUM` or `GENERALIZED_NONCOMMUTATIVE_SUM` expression be writable (24.2.1) to the result output iterator.

§ References

§ Informative References

[P0452r0]

Bryce Adelstein Lelbach. [Binary transform_reduce\(\): The Missing Overload](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0452r0.html). 14 October 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0452r0.html>